Odoo Migration Cookbook — Volume II

Closing the Gaps: Version 19 to Version 20

COMMUNITY EDITION AND ENTERPRISE EDITION · COMPANION EDITION · OCTOBER 2026

An independent companion to Volume I

Every entry verified against the odoo/odoo source trees, branches 19.0 and 20.0

Volume I collected the 19 → 20 replacements used most often in custom modules: the `ir.access.csv` security file, ORM and Python renames, search domains, binary and report handling, view and QWeb xpaths, Owl 3, and the accounting, stock, HR and website mappings. This companion volume deliberately does not repeat them.

Volume II covers the ground Volume I leaves open: the new Python floor and package layout, the moved import surface (`odoo.osv` , `odoo.tools` re-exports), caching and query objects, the access-check and recursion APIs, the `read_group` signature overhaul, field and domain breaking changes, and the rewritten testing toolkit.

Method

- Every entry was verified on 2026-10-11 by comparing the `odoo/odoo` branches `19.0` and `20.0` file by file (release.py, orm package, tools, tests, base and mail models). The verification log is in Appendix A.
- Entries hold for Community and Enterprise alike unless marked **EE**. Enterprise-only modules are out of scope for this volume.
- Code samples are signatures or minimal patterns copied from the 20.0 tree, not full implementations.

How to read the tables

Left column: what your 19 module contains today. Right column: what 20 accepts. A `monospace chip` marks code; bold marks keywords.

Contents

Select a chapter to open it.

1. Runtime, packaging, and the import surface
2. Caching, queries, and SQL objects
3. Model and access API deltas
4. Fields and domains: breaking changes
5. Testing and CI toolkit
6. Grep-driven migration checklist
7. Automated codemods: odoo-bin upgrade_code
8. Appendix A — verification log

1. Runtime, packaging, and the import surface

BACK TO CONTENTS

Python floor moves to 3.12

Odoo 19 runs on Python ≥ 3.10; Odoo 20 raises the floor to **Python ≥ 3.12** (`odoo/release.py` , `MIN_PY_VERSION`). Core code now uses PEP 695 generic syntax, which only 3.12+ parses:

```
# odoo/addons/base/models/ir_config_parameter.py — 20.0
def get_int[T](self, key: str, default: T = 0) -> int | T:
    ...
```

- Upgrade CI images, Dockerfiles and `python_requires` of your addons before attempting the registry upgrade.
- 3.12 also removes `distutils` and tightens `ast` / `inspect` behaviour; audit legacy setup scripts.

Package layout

Odoo 19	Odoo 20
`odoo/http.py` (single module)	`odoo/http/` (package). `import odoo.http` and `from odoo import http` keep working.
`odoo/osv/` with `expression.py`	Removed. Domains live in `odoo/orm/domains.py` (Chapter 4).
`odoo/models` , `odoo/fields` , `odoo/api` shim packages	Unchanged shims re-exporting `odoo.orm.*` . `from odoo import models, fields, api` stays canonical in both versions.
`odoo/orm/` core (models, fields, domains, …)	Same package plus new modules `cache.py` , `models_cached.py` , `query.py` .

The odoo.tools export surface

Several names disappeared from `odoo/tools/__init__.py` in 20, and binary/SQL names became public:

Name in 19 (from odoo.tools)	Status in 20
`Query`	No longer exported. Import `from odoo.orm.query import Query`.
`ormcache`	No longer exported from tools. Use `@api.ormcache` (Chapter 2).
`ormcache_context`	Removed with no replacement.
`convert_xml_import` / `convert_csv_import` / `convert_sql_import`	No longer exported; see internals of `odoo/tools/convert.py`.
—	Newly public: `SQL`, `BinaryValue`, `BinaryBytes`, `drop_view_if_exists`. The binary classes are defined in `odoo/tools/binary.py` and were private in 19.

```
# 19
from odoo.tools import Query, ormcache
from odoo.tools.binary import BinaryValue      # private path

# 20
from odoo import api
from odoo.orm.query import Query
from odoo.tools import BinaryValue, BinaryBytes, SQL
```

2. Caching, queries, and SQL objects

BACK TO CONTENTS

ormcache moves to the api module

The decorator class lives in `odoo/orm/cache.py` in 20 (it was in `odoo/tools/cache.py` in 19) and is re-exported through `odoo.api`:

```
# 19
from odoo.tools import ormcache, ormcache_context

class MyModel(models.Model):
    @ormcache('name')
    def _get_id_by_name(self, name): ...

# 20
from odoo import api

class MyModel(models.Model):
    @api.ormcache('name')            # or: @api.ormcache(cache='stable')
    def _get_id_by_name(self, name): ...
```

- `ormcache_context` is gone. Fold the context values you cached on into explicit method arguments so they become part of the cache key.
- Cache introspection moved with it: `odoo.orm.cache.get_cache_size`, `get_cache_key_counter`, `log_ormcache_stats`.
- Invalidation also changed name: the `registry.clear_cache()` calls scattered in 19 code become `self.env.transaction.invalidate_ormcache()` in 20 — exactly what the bundled codemod `19.4-00-ormcache-on-transaction` rewrites (Chapter 7).

Query and SQL

`Query` (the ORM query builder) is defined in `odoo/orm/query.py` in 20; in 19 it reached you through the `odoo.tools` re-export. The `SQL` composable

class is unchanged in `odoo/tools/sql.py` and is now a public `odoo.tools` export.

```
# odoo/sql_db.py — 20.0
def execute(self, query: SQL | typing.LiteralString | psql.Composable,
            params=None, log_exceptions: bool = True) -> None:
    if isinstance(query, SQL):
        assert params is None, "Unexpected parameters for SQL query object"
```

- Plain string queries still execute (`LiteralString`), but pairing an `SQL` object with `params` is now an assertion error — build parameters into the `SQL` composable.

3. Model and access API deltas

BACK TO CONTENTS

Signatures below were read from `odoo/orm/models.py` on both branches. Volume I covers `check_access_rights` → `check_access` / `has_access`; the remaining renames are listed here.

Odoo 19	Odoo 20
`check_access_rule(operation)`	Removed. Record rules are enforced inside `check_access(operation)` / `has_access(operation)`.
`_filter_access_rules(op)` / `_filter_access_rules_python(op)`	Internal successor `_access_domain(op)` returns the rule domain; the Python-side filter method is gone. Treat both as private.
`check_field_access_rights(fields, operation, ...)`	`check_field_access(field, operation)` and `has_field_access(field, operation)` — per-field checks with `Literal['read','write']`.
`toggle_active()`	Removed with the `toggle_active` window action; call `action_archive` / `action_unarchive` (Volume I, Ch. 2).
`_check_recursion()` / `_check_m2m_recursion(field)`	`_has_cycle(field_name=None)` — already present in 19, so adopt it before migrating.
`copy_translations(records)`	Removed; translation copy is handled internally, no public replacement.
`read_group(domain, fields, groupby, offset=0, limit=None, orderby=False, lazy=True)`	`read_group(domain, groupby=(), aggregates=(), having=(), offset=0, limit=None, order=None)` returning `list[tuple]`.

READ_GROUP BEFORE AND AFTER

```
# 19 — lazy dict-based API
groups = env['sale.order'].read_group(
    domain, ['amount_total'], ['state'], lazy=False)

# 20 — explicit groupby + aggregate specs, tuples back
rows = env['sale.order'].read_group(
    domain,
    groupby=['state'],
    aggregates=['amount_total:sum', '__count'])
for state, total, count in rows: ...
```

- Aggregate specs use the `field:agg` syntax (`sum`, `avg`, `min`, `max`, `array_agg`, …); `__count` replaces the old `state_count` key.
- The `having` parameter accepts a domain on aggregated values — new in the public API.

4. Fields and domains: breaking changes

BACK TO CONTENTS

group_operator is gone — use aggregator

19 still accepts `group_operator` but warns **"Since Odoo 18, 'group_operator' is deprecated, use 'aggregator' instead"**. In 20 the keyword no longer exists:

```
# 19 (DeprecationWarning)          # 20
total = fields.Monetary(          total = fields.Monetary(
    group_operator='sum')             aggregator='sum')
```

Raw SQL in domain leaves now raises

Passing an `SQL` object as a domain leaf value warns in 19 (`"Since 19.0, use Domain.custom(to_sql=…)"`) and raises `TypeError` in 20 (`"Unexpected SQL in condition …, use Domain.custom() instead"`):

```
# 20 — odoo/orm/fields.py docstring pattern
domain = Domain.custom(
    to_sql=lambda table: SQL(
        "(%s IS NULL OR %s = %s)",
        table.company_id, table.company_id, company_id))
```

odoo.osv.expression → odoo.orm.domains

Odoo 19 (odoo.osv.expression)	Odoo 20 (odoo.orm.domains / Domain)
`from odoo.osv import expression`	Import error — the `odoo/osv` package is removed. Use `from odoo.orm.domains import Domain` .
`expression.AND(ds)` / `expression.OR(ds)`	`Domain.AND(ds)` / `Domain.OR(ds)` , or the `&` / `|` operators.

Odoo 19 (odoo.osv.expression)	Odoo 20 (odoo.orm.domains / Domain)
`expression.is_false(model, domain)`	`Domain(d).is_false()` / `.is_true()`, plus `Domain.TRUE` / `Domain.FALSE`.
`expression.normalize_domain(d)`	Internal: `Domain` objects normalize on construction.
`expression.combine(op, unit, zero, ds)`	Use Python operators on `Domain` instances.
`expression.distribute_not`, `prettify_domain`	Removed (were internal helpers).

5. Testing and CI toolkit

BACK TO CONTENTS

Registry test mode is a context manager

```
# 19
self.registry_enter_test_mode()
try:
    ...
finally:
    self.registry_leave_test_mode()

# 20
with self.registry_test_mode():
    ...
```

Query assertions

- `assertQueriesContain` is removed; `assertQueries` and `assertQueryCount` remain.
- New matcher `QueryLike` joins the family `Like` / `WhitespaceInsensitive` / `Approx` for parameterized or reformatted SQL expectations.

MockHTTPClient — new in 20

A composable context manager that patches `requests.sessions.Session.send`:

```
from odoo.tests.common import MockHTTPClient

with MockHTTPClient('POST', 'https://api.example.com/v1/items',
                    return_json={"id": 1}) as mock:
    self.env['my.integration'].push_items()
mock.assert_called_once()
```

- Matching: `method`, `url` (host+path prefix, query optional), or a `matcher(request)` predicate for headers/body.
- Responses from `return_json` (sets `application/json`) or `return_body`; `return_status` / `return_headers` accept callables. Mocks nest: unmatched requests fall through to the outer mock or real `requests`.
- Call assertions: `mock.assert_called(n_times)`, `assert_called_once()`, `assert_not_called()`.

HttpCase additions

Helper (20.0)	Purpose
`self.session`	Current `Session` of the test browser (new; 19 had no accessor).
`update_session(**items)` / `update_session_context(**items)`	Mutate and persist session values (and its context) between requests.
`csrf_token()`	Fetch a CSRF token for forged form posts.
`file_read(path)`	Classmethod returning file contents as `BinaryBytes` — pairs with the binary API of Volume I Ch. 4.
`mock_datetime_and_now(dt)`	Patches `datetime` helpers and `env.cr.now()` together, for sync-style tests.
`response.raise_for_status()`	Overridden to surface the Odoo error payload in the exception text.

6. Grep-driven migration checklist

BACK TO CONTENTS

Run these greps over every custom addon; the right-hand side is the 20-clean replacement (volume reference where the topic is covered in depth).

grep pattern	action in 20	see
`group_operator=`	Rename kwarg to `aggregator=`	II.4
`check_access_rights(`	`has_access(` or `check_access(`	I.2
`check_access_rule(`	`check_access(`	II.3
`check_field_access_rights(`	`has_field_access(` / `check_field_access(`	II.3
`toggle_active`	`action_archive` / `action_unarchive`	I.2
`_check_recursion(`	`_has_cycle(`	II.3
`from odoo.osv import`	`from odoo.orm.domains import Domain`	II.4
`tools import ormcache`	`api.ormcache`	II.2
`ormcache_context`	Drop; explicit cache-key arguments	II.2
`tools import Query`	`odoo.orm.query import Query`	II.2
`convert_xml_import`	Internal only; reimplement via `odoo.tools.convert` APIs	II.1
`read_group(` with `lazy=`	New `groupby/aggregates` signature	II.3
`type='json'` route	`type='jsonrpc'`	I.5
`ir.model.access.csv`	`ir.access.csv`	I.1
`assertQueriesContain`	`assertQueries` + `QueryLike`	II.5
`registry_enter_test_mode`	`registry_test_mode()` context	II.5

grep pattern	action in 20	see
`copy_translations(`	Delete call; no replacement	II.3
`pytz`	`datetime.UTC` / `zoneinfo`	I.2

7. Automated codemods: odoo-bin upgrade_code

BACK TO CONTENTS

Odoo ships a codemod repository at `odoo/upgrade_code/`. Each script is named `{version}-{seq}-{name}.py` and exposes `upgrade(file_manager)`; the `upgrade_code` CLI command (present in both 19 and 20, also runnable standalone)
rewrites
your
addon sources in place — `.py`, `.js`, `.css`, `.scss`, `.xml`, `.csv`, `.po`, `.pot`.

Running it

```
# all scripts for the 19 → 20 span, on your addons
odoo-bin upgrade_code --from 19.0 --addons-path /path/to/addons

# a single script (substring match is accepted)
odoo-bin upgrade_code --script ir-access --addons-path /path/to/addons

# standalone, without odoo-bin (resolved paths required)
python3 odoo/cli/upgrade_code.py --from 19.0 --addons-path /path/to/addons
```

- `--from` and `--script` are mutually exclusive and one is required; `--to` defaults to the running release and both bounds are inclusive — selection is by the version prefix of the script file name.
- `--glob` narrows the file set; `--dry-run` prints the `updated:` / `deleted:` list without writing. The command exits with code 1 when any file changed, which makes it usable as a CI gate.
- The docstring is explicit: scripts are best-effort helpers for the heavy lifting, **not silver bullets** — review the diff in a git checkout before upgrading the registry.

The script contract (write your own)

```
def upgrade(file_manager):
    files = [f for f in file_manager if f.path.suffix == '.py']
    for fileno, file in enumerate(files, start=1):
        content = file.content            # lazy read
        file.content = content.replace('old_api(', 'new_api(')
        file_manager.print_progress(fileno, len(files))
```

- File objects expose `path` (pathlib), `addon` and a lazy `content` property; assigning an unchanged value is a no-op, assigning `None` deletes the file on save.
- The manager adds `get_modules()`, `get_file(module, name)`, `add_to_summary(msg)` (printed at the end) and `print_progress(current, total, file_name)`.
- Custom scripts must live inside `odoo/upgrade_code/` — `--script` resolves candidates there and refuses paths outside it, so drop your team rewrites into that directory (or call `migrate()` from Python).

Scripts selected for a 19 → 20 run

Script	What it rewrites
`17.5-01-tree-to-list`	View mode `tree` → `list` in `view_mode`/binding fields and archs (the rename already landed in 19).
`18.1-00-sql-constraint`	`_sql_constraints = [...]` → `models.Constraint(...)` assignments (Volume I, Ch. 2).
`18.1-02-route-jsonrpc`	Route kwarg `type='json'` → `type='jsonrpc'` (Volume I, Ch. 5).
`18.2 / 18.3 l10n scripts`	Chart-template records and fiscal-position tax mappings (`tax_src_id` / `tax_dest_id` → alternative-tax columns).
`18.5-00-domain-dynamic-dates`	Inlines unary-minus constants (`-1`) inside domains; `18.5-00-no-tax-tag-invert` fixes tax `formula` signs; `18.5-00-deprecated-properties` cleans deprecated property fields.

Script	What it rewrites
`19.1-00-t-call`	Hoists `t-set` nodes ahead of the `t-call` / `t-snippet-call` that consumes them — Owl 3 evaluation order (Volume I, Ch. 9).
`19.3-00-account-groups`	`account.group` CSV/data → `account.account` hierarchy with a computed `parent_id` column (Volume I, Ch. 10).
`19.3-00-account-report-foldable`	Normalizes the `foldable` value of account report XML nodes to a boolean.
`19.3-00-base64-in-xml`	Migrates inline `type="base64" … file=` XML payloads to the 20 loading form.
`19.4-00-ir-access`	Merges `ir.model.access.csv` plus `<record model="ir.rule">` XML into `security/ir.access.csv` with `crud` operation strings and normalized domains (Volume I, Ch. 1).
`19.4-00-ormcache-on-transaction`	`registry.clear_cache` → `transaction.invalidate_ormcache` (this chapter's sibling, Ch. 2).
`19.5-00-tuple-rec_names_search`	`_rec_names_search = ['name']` → tuple form.
`20.0-00-search-date-filters`	Drops `start_month` / `end_month`-style range attributes from `date` search filters — no longer parsed, rejected by the RNG.

- `owl3-migration.py`, `tools_etree.py` and `tools_js_expressions.py` carry no version prefix: they never enter a `--from/--to` range. The first is the Owl 3 JavaScript migration to trigger explicitly with `--script owl3-migration`; the other two are helper libraries the scripts import (`from odoo.upgrade_code.tools_etree import …`).

Suggested workflow

- Run `upgrade_code --from 19.0 --dry-run` first; then the real run on a clean git branch; review `git diff` file by file.
- Commit the codemod result **before** the registry upgrade (`-u`): the scripts fix source code, not database rows — data upgrades remain the job of each module's `pre/post` migration scripts.
- Re-run your test suite; the codemods cannot judge semantics (e.g. an `ir.rule` domain that 20 validates differently, Volume I, Ch. 1).

8. Appendix A — verification log

BACK TO CONTENTS

Branches compared on 2026-10-11: `odoo/odoo@19.0` and `odoo/odoo@20.0`.

- `odoo/release.py` — `MIN_PY_VERSION` (3,10) vs (3,12).
- `odoo/` directory listings — `osv` present in 19 only; `http.py` → `http/`; `orm/` gains `cache.py`, `models_cached.py`, `query.py`.
- `odoo/tools/__init__.py` — export diff (Query, ormcache, convert_* removed; SQL, BinaryValue, BinaryBytes, drop_view_if_exists added).
- `odoo/orm/models.py` — method-set diff (access checks, `_has_cycle`, `read_group` signatures).
- `odoo/orm/fields.py` — `group_operator` warning in 19, absence in 20; `Domain.custom` deprecation → `TypeError`.
- `odoo/orm/domains.py` vs `odoo/osv/expression.py` — helper mapping.
- `odoo/tests/common.py` — test-mode context manager, `MockHTTPClient`, `QueryLike`, HttpCase session helpers.
- `odoo/sql_db.py` — `execute()` signature accepting `SQL | LiteralString | Composable`.
- `odoo/cli/upgrade_code.py` and the `odoo/upgrade_code/` script list on both branches — command flags, file-manager contract and per-script behaviour quoted in Chapter 7.

Enterprise-only deltas (payroll, budget, field service, helpdesk) are not verifiable from the public tree and are intentionally left to Volume I's mapping tables and your Enterprise source review.